

Lab 3D: Subagents & Agent Teams — Parallel AI Work

Duration: 45 minutes

Difficulty: Advanced

Continues from: Lab 3C (dashboard project with commands, hooks, and MCP)

© 2026 AIA Copilot — Instructor-Led Training Material

Overview

Everything so far has been one Claude session doing one thing at a time. Subagents change that — you can spin up specialized AI assistants that work in parallel, each with their own role, tools, and model. This is how production teams scale AI-assisted development.

What You'll Build

- Use built-in subagents (Explore, Plan)
 - Create a security auditor subagent with read-only access
 - Create a documentation generator subagent
 - Orchestrate multiple subagents working in parallel on your dashboard
-

Setup

Continue in your dashboard project:

```
cd business-dashboard
claude
```

Clear Context

```
/clear
```

Phase 1: Built-in Subagents

Claude Code ships with built-in agents. Discover them:

```
/agents
```

You should see at least `explore` and `plan`. Try the explore agent:

```
@explore What testing framework does this project use?  
What is the project structure? How are routes organized?
```

Notice three things: 1. **Speed** — explore uses Haiku, which is fast and cheap 2. **Read-only** — it can only read files, not modify them 3. **Focused** — it starts a separate session scoped to your question

Now try with a more complex exploration:

```
@explore Trace the request flow from a POST to /api/tasks through  
to the database and back. What middleware runs along the way?
```

Compare this to asking the same question in your main session. The subagent stays focused on exploration without cluttering your main conversation with file reads.

Phase 2: Create a Security Auditor

Build a specialized subagent that audits your code for vulnerabilities:

```
Create .claude/agents/security-auditor.md with:
- name: security-auditor
- description: Security-focused code reviewer for vulnerability scanning
- tools: Read, Grep, Glob, Bash(npm audit:*)
- model: sonnet
- Instructions to check for: dependency vulnerabilities (npm audit),
  hardcoded secrets, SQL injection, command injection, XSS,
  and missing input validation
- Report format: CRITICAL (red), WARNING (yellow), INFO (green)
  with specific file locations and recommended fixes
```

Restart Claude to load the new agent, then invoke it:

```
/quit
```

```
claude
```

```
@security-auditor audit this entire project
```

Review the findings. Notice: - The auditor ran `npm audit` (the only bash command it's allowed) - It searched for secrets with Grep - It CANNOT fix the issues — it can only report (read-only tools)

Now ask it to fix something:

```
@security-auditor fix the SQL injection vulnerability you found
```

It should fail or refuse — it doesn't have Write or Edit tools. That's the principle of least privilege in action.

Phase 3: Create a Documentation Generator

Build a subagent that writes documentation:

```
Create .claude/agents/doc-writer.md with:
```

- name: doc-writer
- description: Generates and updates project documentation
- tools: Read, Grep, Glob, Write
- model: haiku
- Instructions to: analyze code structure, generate README sections, document API endpoints with request/response examples, and create a getting-started guide
- Output format: clean markdown suitable for GitHub

Restart and invoke:

```
@doc-writer Generate complete API documentation for all endpoints  
in this project. Include curl examples for each.
```

Notice: doc-writer uses **Haiku** (cheapest model). Documentation generation doesn't need Opus-level reasoning — it's reading code and formatting output. Match the model to the task.

Phase 4: Orchestrate Multiple Agents

This is where it gets powerful. Use your main session to coordinate:

```
@security-auditor audit the src/routes directory for any input validation issues
```

Wait for the report. Then:

```
Based on the security audit above, fix all the input validation issues  
the auditor found. Add proper validation to every endpoint that accepts user input.
```

You just used a read-only auditor to identify problems, then your main session (with full write access) to fix them. The auditor can't accidentally change code while scanning, and your main session gets a focused list of exactly what to fix.

Phase 5: Subagent Decision Framework

Ask Claude to help you internalize the framework:

When should I use my main session vs a subagent?
Give me a decision rule I can apply in 5 seconds.

The rule:

Use Main Session When...	Use Subagent When...
You need to modify files	Task is read-only inspection
Task requires full project context	Task is focused on one question
You're doing sequential work	You want faster/cheaper results
Changes depend on conversation history	You want a fresh perspective

Phase 6: Agent Teams — True Parallel Work

Subagents are child sessions that report back to you. **Agent Teams** are something different — multiple independent Claude Code instances working simultaneously on separate branches of the same project. This is production-grade parallelism.

Explain the difference between subagents and agent teams in Claude Code.
When would I use each?

The key differences:

Feature	Subagents	Agent Teams
Instances	Child within your session	Independent Claude sessions
Context	Inherits from parent	Own CLAUDE.md, MCP, skills
Isolation	Shares project directory	Each gets own git worktree
Coordination	Reports back to parent	Merge via git at the end
Cost	Same session budget	Each teammate uses own tokens
Best for	Focused read-only tasks	Parallel feature development

Set up a git worktree for parallel work:

```
Create a git worktree for a "feature/dashboard-charts" branch.
Show me the command and explain what it does to the file system.
```

Now spin up an agent team:

```
Explain how to use the --agent-team flag or the Claude Code agent team feature
to spin up teammates working on parallel tasks. Show me an example that:
1. Creates a teammate to add chart visualizations on one branch
2. Creates a teammate to add user authentication on another branch
3. Both working simultaneously
```

The workflow: 1. Main session defines tasks and creates branches 2. Each teammate gets their own worktree (isolated copy of the code) 3. Teammates load their own CLAUDE.md, MCP servers, and skills automatically 4. Teammates work in parallel — no conflicts because they're on separate branches 5. When done, merge branches through normal git pull requests

```
Show me the git worktree list to see all active worktrees.
Then show how to clean up worktrees when work is complete.
```

Important cost consideration: Agent teams use cheaper models by default, but each teammate consumes its own token budget. A team of 3 working for 10 minutes might cost \$5-15 depending on the models. Always scope tasks tightly to control costs.

Phase 7: Git Worktrees — The Foundation of Parallel AI

Agent teams use git worktrees under the hood. Understanding worktrees directly gives you more control:

```
Create a worktree for "feature/dashboard-api-docs" and explain what happened
in the filesystem. Where does the new worktree live? How does git track it?
```

Open a second terminal and start Claude Code in the worktree:

```
cd ../business-dashboard-feature-dashboard-api-docs
claude
```

Now you have two independent Claude sessions on two branches. Give the second session a task:

```
Generate complete API documentation for all endpoints in this project.
Create a docs/ directory with a README and individual doc files per route.
```

Meanwhile, your first session can keep working on other features. When both are done, merge through normal git:

```
Switch back to main branch. Show the branches that have worktree changes.
Merge the api-docs branch with a merge commit.
```

Clean up:

```
Remove the worktree for feature/dashboard-api-docs and delete the branch
```

What You Learned

- **Built-in agents** (explore, plan) handle common tasks cheaply and quickly
- **Custom agents** have defined roles, restricted tools, and appropriate models
- **Tool restrictions** enforce least-privilege — auditors can't modify, writers can't execute
- **Model selection matters** — Haiku for simple tasks, Sonnet for analysis, Opus for complex reasoning
- **Orchestration pattern**: specialized subagents identify → main session implements

- **Agent Teams** enable parallel development with isolated branches and independent context
- **Git worktrees** are the foundation — each team member gets a clean working copy

Subagents vs Agent Teams: The Decision Rule

Use subagents when you need a quick focused inspection without leaving your session. **Use agent teams** when you have multiple features that can be developed in parallel.

The Big Picture

You now have a complete AI development environment: - **CLAUDE.md** defines how Claude works (Day 2) - **Custom commands** encode your team's workflows (Lab 3A) - **Hooks** enforce safety and compliance (Lab 3B) - **MCP** connects to external tools and data (Lab 3C) - **Subagents** handle focused inspections (Lab 3D) - **Agent Teams** parallelize feature development (Lab 3D)

This is not a coding assistant. This is an AI development team.

Congratulations — you've completed all Day 3 labs.

